

Capgemini Sprint Programme

Detailed .NET Core with Azure Study Book

Based on syllabus workbook: D:\Capgemini\Shiv-Assignments\Syllabus.xlsx

Generated: 2026-05-04

This is a study-book style reference. It is organized by concepts, not by repository folders. It covers the syllabus deeply and adds related professional topics useful for Sprint implementation, evaluations, interviews, and OCEAN L1 preparation.

How to Study This Book

- Read the main chapters first to build conceptual understanding.
- Use the syllabus focus sheets for day-by-day coverage without treating them as a folder or submission tracker.
- For every technical topic, practice one tiny working example and one spoken explanation.
- Before assessment, revise the self-test questions, code snippets, commands, and interview angles.

Curriculum Coverage Summary

.NET 8.0 and C# 12.0	20
Azure PaaS Services	9
Cloud Networking & Storage	8
ASP.NET Core Web API	7
ASP.NET Core MVC	6
Programming Foundation and Problem Solving	5
RDBMS & SQL Server	4
Web Basics (HTML5, CSS 3, JavaScript)	4
Cloud Computing & Azure Fundamentals	4
Entity Framework Core	3
Unit Testing Framework (XUNIT, NUnit)	2
LINQ	2
Git	2
Power Skill 1	1
Power Skill 2	1
Power Skill 3	1
Azure DevOps	1
Power Skill 4	1
Power Skill 5	1

Core Learning Path

- .NET and C# fundamentals create the language base.
- Problem solving, testing, SQL, LINQ, and EF Core create backend confidence.
- HTML/CSS/JavaScript, MVC, and Web API create application-building ability.
- Azure DevOps, cloud fundamentals, networking, storage, PaaS, and data services create deployment readiness.
- Git, power skills, Sprint implementation, OCEAN L1, and Gen AI prep complete workplace readiness.

Part 1 - .NET 8, .NET Core Architecture, and C# 12

The .NET Platform

- .NET is a developer platform for building console apps, web apps, APIs, cloud services, desktop apps, mobile apps, background workers, and libraries.
- .NET 8 is a long-term support release. In training, focus on SDK commands, project structure, runtime behavior, dependency management, and cross-platform execution.
- The SDK includes compilers, templates, build tools, and CLI commands. The runtime executes compiled applications. The base class library provides common APIs for strings, collections, file I/O, networking, JSON, threading, and security.
- .NET Framework is Windows-only and older. Modern .NET is cross-platform, modular, open-source, and optimized for cloud-native development.

```
dotnet --info
dotnet new console -n DemoApp
dotnet run
dotnet build
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Project Files and Execution

- A .csproj file declares target framework, SDK type, package references, nullable settings, implicit usings, and build metadata.
- Program.cs is the application entry point. In ASP.NET Core, it also configures services and middleware.
- Build produces assemblies. Run executes the app. Publish prepares deployment output for a target runtime and hosting model.
- NuGet packages extend the app. Always keep package references intentional and avoid adding packages for tiny problems already solved by the framework.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

C# Type System

- Value types store data directly and include int, double, bool, DateTime, enum, and struct types. Reference types store references to objects and include class, string, arrays, delegates, and interfaces.
- Boxing converts a value type to object; unboxing converts back. It can allocate memory and fail at runtime if the wrong type is used.
- Nullable types handle missing values. int? means int or null. Nullable reference types are compile-time warnings that help avoid NullReferenceException.
- var is statically typed after inference. dynamic delays binding until runtime and should be used rarely.

```
int number = 10;
object boxed = number;
int unboxed = (int)boxed;

var name = "Shiv";      // string at compile time
dynamic value = 10;     // runtime binding
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

C# 12 and Modern Language Features

- C# 12 adds features such as primary constructors, collection expressions, default lambda parameters, aliasing any type, and other improvements that reduce boilerplate.
- Modern C# favors expressive, safe, concise code. Use pattern matching, null-coalescing, object initializers, records where useful, and collection expressions when they improve readability.
- Do not chase syntax for its own sake. In interviews, explain how a feature improves clarity, type safety, or maintainability.

```
public class Employee(string name, decimal salary)
{
    public string Name { get; } = name;
    public decimal Salary { get; } = salary;
}
```

```
int[] marks = [80, 85, 90];
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 2 - C# OOP, Exceptions, Collections, Delegates, and Async

Object-Oriented Programming

- A class defines state and behavior. An object is an instance of a class. Encapsulation keeps data and operations together while hiding internal details.
- Inheritance models specialization. Polymorphism lets code depend on a base type or interface while runtime objects provide specific behavior.
- Abstraction exposes essential behavior and hides implementation. Interfaces and abstract classes are common abstraction tools.
- Access modifiers control visibility: private, public, protected, internal, protected internal, and private protected.

```
public interface IEmployeeService
{
    Task<Employee?> GetByIdAsync(int id);
}

public class EmployeeService : IEmployeeService
{
    public Task<Employee?> GetByIdAsync(int id) => Task.FromResult<Employee?>(null);
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Constructors, Static Members, and Object Lifetime

- Constructors initialize objects. Overloaded constructors support different creation paths, but too many constructors can make the API confusing.
- Static members belong to the type rather than an instance. Use static for pure helpers or shared constants, not for per-user web state.
- Destructors/finalizers are rarely used directly. IDisposable and using statements are the normal way to release unmanaged resources deterministically.

```
using var stream = File.OpenRead("data.txt");
// stream.Dispose() is called automatically at the end of scope.
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Exception Handling

- Exceptions represent abnormal flow. Use try/catch/finally when you can recover, add context, translate to a user/API response, or guarantee cleanup.
- Throw preserves error signaling. Use throw; instead of throw ex; when rethrowing to preserve stack trace.
- Custom exceptions should add meaning. Do not create custom exception types for every small validation error.
- In Web API projects, prefer global exception handling middleware for consistent ProblemDetails responses.

```
try
{
    var age = int.Parse(input);
}
catch (FormatException ex)
{
    logger.LogWarning(ex, "Invalid age input");
}
finally
{
    // cleanup if needed
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Collections, Delegates, Events, and Lambdas

- List<T> is a dynamic array. Dictionary<TKey,TValue> provides fast key lookup. HashSet<T> stores unique values. Queue<T> and Stack<T> model FIFO and LIFO.
- Delegates store references to methods. Events use delegates to notify subscribers without tightly coupling publisher and receiver.
- Lambda expressions are concise functions. LINQ, event handlers, and asynchronous callbacks use them heavily.
- Expression trees represent code as data and are used by LINQ providers such as EF Core to translate queries to SQL.

```
var adults = people
    .Where(p => p.Age >= 18)
    .OrderBy(p => p.LastName)
    .ToList();
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Threading, Tasks, Async, and Await

- Threads are low-level execution units. Tasks represent asynchronous operations and are easier to compose.
- Use async/await for I/O: database calls, file reads, HTTP calls, queue operations, and cloud SDK calls.
- Do not block async code with .Result or .Wait in ASP.NET apps. It can hurt scalability and sometimes deadlock.
- Locks protect shared mutable state, but web apps should usually avoid shared mutable state when possible.

```
public async Task<Employee?> GetEmployeeAsync(int id)
{
    return await _context.Employees.FindAsync(id);
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 3 - Programming Foundation and Problem Solving

Data Structures

- Choose a data structure based on operations. Arrays are fast for indexing, linked lists are flexible for inserts if you already have a node, stacks reverse order, queues preserve arrival order, trees represent hierarchy, and graphs represent networks.
- Strings are sequences of characters but are immutable in C#. Use `StringBuilder` for repeated modification.
- Graphs require careful representation. Adjacency lists are memory efficient for sparse graphs; adjacency matrices are simple but memory-heavy.

```
Stack<int> stack = new();
stack.Push(10);
stack.Push(20);
int latest = stack.Pop();
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Searching and Sorting

- Linear search works on any collection and is $O(n)$. Binary search is $O(\log n)$ but requires sorted input.
- Bubble, selection, and insertion sort are simple $O(n^2)$ algorithms. They are useful for learning but not ideal for large data.
- Merge sort and quick sort are common advanced sorting algorithms. Merge sort is stable; quick sort is often fast but pivot choice matters.
- Always ask about constraints. The best algorithm for 20 items may not be best for 2 million items.

```
int BinarySearch(int[] arr, int target)
{
    int left = 0, right = arr.Length - 1;
    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Recursion

- Recursion solves a problem by reducing it to smaller versions of itself.
- Every recursive method needs a base case and a recursive case. Without a base case, recursion continues until stack overflow.
- Tail recursion performs the recursive call as the final step. Tree recursion branches into multiple recursive calls. Indirect recursion occurs when functions call each other.
- For production C#, iterative solutions may be safer for very deep input because .NET does not guarantee tail-call optimization in ordinary code.

```
int Factorial(int n)
{
    if (n <= 1) return 1;
    return n * Factorial(n - 1);
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 4 - Unit Testing with xUnit and NUnit

Testing Mindset

- A good test documents expected behavior. It should be deterministic, isolated, readable, and fast.
- Use Arrange, Act, Assert. Arrange creates input and dependencies, Act calls the method, Assert verifies output or side effects.
- Unit tests are not a replacement for integration tests. Unit tests check small behavior; integration tests verify real wiring such as database or API configuration.

```
[Fact]
public void Add_ReturnsSum()
{
    var calc = new Calculator();
    var result = calc.Add(2, 3);
    Assert.Equal(5, result);
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Parameterized Tests, Fixtures, and Mocking

- Parameterized tests run the same test logic with multiple inputs. In xUnit, use [Theory] and [InlineData]. In NUnit, use [TestCase].
- Fixtures share expensive setup between tests. Use them carefully so tests do not accidentally depend on shared mutable state.
- Mocking and stubbing replace dependencies such as repositories, HTTP clients, email senders, or cloud services.
- Parallel test execution improves speed but can expose shared-state problems.

```
[Theory]
[InlineData(2, 3, 5)]
[InlineData(-1, 1, 0)]
public void Add_ReturnsExpected(int a, int b, int expected)
{
    Assert.Equal(expected, new Calculator().Add(a, b));
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 5 - RDBMS, SQL Server, and T-SQL

Database Fundamentals

- A relational database stores data in tables and links tables through keys. Primary keys uniquely identify rows; foreign keys enforce relationships.
- Data integrity includes entity integrity, referential integrity, domain integrity, and user-defined rules.
- Normalization reduces duplication and update anomalies. Understand 1NF, 2NF, and 3NF well enough to apply them in examples.

```
CREATE TABLE Authors (  
    AuthorId INT IDENTITY PRIMARY KEY,  
    Name NVARCHAR(100) NOT NULL  
);
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

DDL, DML, DCL, and Joins

- DDL defines schema: CREATE, ALTER, DROP. DML manipulates data: SELECT, INSERT, UPDATE, DELETE. DCL controls permissions: GRANT, DENY, REVOKE.
- INNER JOIN returns matching rows. LEFT JOIN keeps all rows from the left table. RIGHT JOIN keeps rows from the right table. FULL JOIN keeps all rows from both.
- Subqueries can be scalar, row-based, or table-based. Use them when a query depends on another query result.

```
SELECT b.Title, a.Name AS AuthorName  
FROM Books b  
INNER JOIN Authors a ON a.AuthorId = b.AuthorId  
WHERE b.AvailableCopies > 0;
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Views, Indexes, Stored Procedures, and Execution Plans

- Views save query definitions and can simplify reporting or security. They are not automatically performance magic.
- Indexes speed read queries but slow writes and consume storage. Index columns used in joins, filters, and ordering.
- Stored procedures package database logic with input/output parameters. Use TRY-CATCH in T-SQL for controlled error handling.
- Execution plans show how SQL Server executes a query. Learn to spot scans, seeks, joins, and missing index hints.

```
CREATE PROCEDURE GetBooksByAuthor  
    @AuthorId INT  
AS  
BEGIN  
    SELECT * FROM Books WHERE AuthorId = @AuthorId;  
END;
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 6 - LINQ and Entity Framework Core

LINQ Fundamentals

- LINQ integrates query operations into C#. It supports filtering, projection, ordering, grouping, joining, paging, and aggregation.
- Method syntax is usually more common in production C#. Query syntax can be clearer for joins and grouping.
- IEnumerable queries execute in memory. IQueryable queries can be translated by providers such as EF Core into SQL.

```
var highEarnings = employees
    .Where(e => e.Salary > 50000)
    .OrderByDescending(e => e.Salary)
    .Select(e => new { e.Name, e.Salary })
    .ToList();
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

EF Core Overview and Patterns

- DbContext is the unit of work. DbSet<T> represents a table. Entities are tracked, modified, and saved using SaveChangesAsync.
- Code First uses C# classes and migrations. Database First scaffolds from an existing database.
- Repository pattern can isolate data access. Use it when it reduces duplication or improves testability, not automatically for every simple app.

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Migrations and Data Loading

- Migrations create a history of schema changes. Review migrations before applying them because generated code reflects current model assumptions.
- Eager loading uses Include. Explicit loading loads navigation properties on demand. Lazy loading can be convenient but may hide performance problems.
- Raw SQL and stored procedures are available when LINQ is not expressive enough, but all user input must be parameterized.

```
dotnet ef migrations add AddBooks
dotnet ef database update
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 7 - Web Basics: HTML5, CSS3, JavaScript, DOM, and Validation

HTML Forms and Semantic Structure

- HTML defines document structure. Use semantic elements when they describe meaning: header, nav, main, section, article, aside, footer.
- Forms send user input. GET places data in the URL and suits search/filter. POST sends data in the body and suits creation or state changes.
- Labels improve accessibility and click behavior. Fieldsets and legends group related inputs.

```
<form method="post">
  <label for="email">Email</label>
  <input id="email" name="Email" type="email" required />
  <button type="submit">Save</button>
</form>
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

CSS Layout

- The box model has content, padding, border, and margin. Many layout bugs come from misunderstanding which space belongs to which part.
- Positioning can be static, relative, absolute, fixed, or sticky. Avoid absolute positioning for normal page layout unless necessary.
- Flexbox is best for one-dimensional alignment. Grid is best for two-dimensional layout.

```
.toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

JavaScript, DOM, Events, and Fetch

- JavaScript adds behavior in the browser. Understand variables, data types, functions, hoisting, scope, arrow functions, iterators, and generators.
- DOM APIs let scripts read and modify HTML. Event listeners respond to user actions.
- fetch calls APIs asynchronously and returns promises. Always handle non-success HTTP responses and parse JSON carefully.

```
const response = await fetch('/api/products');
if (!response.ok) throw new Error('Unable to load products');
const products = await response.json();
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 8 - ASP.NET Core MVC and Razor Pages

MVC Request Flow

- A browser request enters middleware, routing selects an endpoint, model binding creates action parameters, filters run, the action executes, and a result returns.
- Controllers should coordinate rather than contain all logic. Move data access and business rules into services or repositories when complexity grows.
- Views should render data, not perform heavy business logic. Use view models that match the screen.

```
public async Task<IActionResult> Details(int id)
{
    var employee = await _service.GetByIdAsync(id);
    return employee is null ? NotFound() : View(employee);
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Routing, Actions, and Data Transfer

- Conventional routing uses patterns like {controller=Home}/{action=Index}/{id?}. Attribute routing declares route templates on controllers/actions.
- Action return types include ViewResult, RedirectToAction, NotFound, File, JsonResult, IActionResult, and ActionResult<T>.
- ViewBag and ViewData are flexible but weakly typed. Strongly typed models and view models are safer for real applications.

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Filters, Identity, JWT, and Deployment

- Filters handle cross-cutting work around actions: authorization, resource checks, action logic, exceptions, and result processing.
- Authentication proves identity. Authorization checks permissions. ASP.NET Core Identity adds users, roles, password hashing, lockout, and token support.
- Deployment requires publish output, hosting target, configuration, database connection, logging, and environment-specific settings.

```
app.UseAuthentication();
app.UseAuthorization();
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Razor Pages vs MVC

- Razor Pages organize around pages and PageModel handlers such as OnGet and OnPost. MVC organizes around controllers and actions.
- Choose Razor Pages for page-focused CRUD/admin workflows. Choose MVC when controller-based grouping or larger separation suits the app.
- Both use Razor syntax, model binding, validation, dependency injection, and middleware.

```
public class EditModel : PageModel
{
    public IActionResult OnGet(int id) => Page();
    public IActionResult OnPost() => RedirectToPage("Index");
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 9 - ASP.NET Core Web API

REST and HTTP

- REST-style APIs expose resources through URLs and standard verbs. GET reads, POST creates, PUT replaces/updates, PATCH partially updates, DELETE removes.
- Status codes are part of the contract: 200 OK, 201 Created, 204 NoContent, 400 BadRequest, 401 Unauthorized, 403 Forbidden, 404 NotFound, 409 Conflict, 500 Server Error.
- Web API differs from MVC views because the response is usually JSON/XML rather than HTML.

```
[HttpGet("{id}")]
public async Task<ActionResult<EmployeeDto>> Get(int id)
{
    var employee = await _service.GetByIdAsync(id);
    return employee is null ? NotFound() : Ok(employee);
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

DTOs, Validation, Formatting, and CORS

- DTOs define request and response shape. They reduce overposting and avoid exposing EF navigation graphs.
- Content negotiation lets clients and servers agree on response format. JSON is the usual default in ASP.NET Core APIs.
- CORS is a browser rule. Configure it only for trusted frontend origins and required methods/headers.
- FluentValidation can keep validation rules clear, testable, and separate from entity classes.

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("Frontend", policy =>
        policy.WithOrigins("https://example.com")
            .AllowAnyHeader()
            .AllowAnyMethod());
});
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Error Handling, Logging, Caching, Versioning, and Testing

- Global error handling keeps error responses consistent. Prefer ProblemDetails for APIs.
- Logging should capture endpoint, request id, user id where appropriate, validation failure, exception, and timing.
- Caching can improve performance for stable data, but stale data is a real risk.
- Versioning protects clients when APIs change. Swagger and Postman help manual testing; unit and integration tests protect regression.

```
return Problem(
    title: "Validation failed",
    statusCode: StatusCodes.Status400BadRequest);
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 10 - Azure DevOps and Cloud Fundamentals

Azure DevOps Components

- Boards manage work items, backlogs, sprints, and task tracking. Repos store source code. Pipelines automate build and release.
- Artifacts store packages. Test Plans manage manual and exploratory testing.
- A healthy team workflow links requirements to commits, builds, deployments, and test evidence.

trigger:

- main

steps:

- script: dotnet restore
- script: dotnet build --configuration Release
- script: dotnet test

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Cloud Computing Basics

- Cloud benefits include elasticity, global reach, managed services, high availability options, and faster provisioning.
- CapEx is upfront capital purchase. OpEx is usage-based operating cost. Cloud shifts many workloads toward OpEx.
- Public cloud is shared provider infrastructure. Private cloud is dedicated to one organization. Hybrid combines both.

Common Azure hierarchy:

Management group -> Subscription -> Resource group -> Resource

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Azure Governance, Identity, and Portal Tools

- Resource groups organize lifecycle and permissions. Azure Resource Manager handles deployment and management.
- Azure Active Directory is now Microsoft Entra ID. It manages identities, users, groups, app registrations, and authentication.
- RBAC controls what identities can do at a scope. Azure Policy enforces rules. Tags support cost and ownership reporting.
- Azure CLI, PowerShell, Cloud Shell, ARM templates, and Bicep help automate management.

```
az group create --name rg-training --location eastus
```

```
az webapp list --resource-group rg-training
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 11 - Azure Networking, Storage, API Management, and CDN

Virtual Machines and VNets

- Azure VMs provide infrastructure control. You manage operating system patching, security, monitoring, backups, and cost.
- VNets provide private address space. Subnets divide it. NSGs filter traffic. ASGs group NICs for security rules.
- Private IP communication should be used inside a VNet. Public exposure should be limited to required ports and trusted sources.

Design example:
VNet 10.0.0.0/16
Subnet web 10.0.1.0/24
Subnet data 10.0.2.0/24
NSG allows HTTPS to web only

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Advanced Networking

- Service endpoints keep traffic to supported Azure services on the Azure backbone while preserving public service endpoints.
- Private endpoints expose services through private IPs and are preferred for stronger private access.
- VNet peering connects virtual networks. VPN Gateway connects Azure networks to on-premises networks.
- Application Gateway is layer 7 load balancing with routing and optional WAF. CDN caches content closer to users.

Common decision:
Use Load Balancer for layer 4 traffic.
Use Application Gateway for HTTP routing and WAF.
Use CDN for static global content acceleration.

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Storage Services

- Blob Storage stores unstructured objects. Azure Files provides SMB/NFS file shares. Queues support asynchronous messaging. Tables store key-value data. Disks back VMs.
- Storage accounts have redundancy options such as LRS, ZRS, GRS, and GZRS. Choose based on durability and cost.
- Use SAS, RBAC, managed identity, private endpoints, encryption, lifecycle rules, and logging for secure storage design.

Blob use cases:
images, documents, backups, logs, exports, static assets

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

API Management

- Azure API Management is a gateway layer for APIs. It can publish APIs, apply policies, transform requests/responses, mock responses, enforce subscriptions, and monitor usage.
- Products group APIs for consumers. Policies can add headers, rate limits, JWT validation, caching, routing, and transformations.
- Self-hosted gateway supports hybrid or on-premises API gateway scenarios.

APIM flow:
Client -> API Management policy pipeline -> Backend API -> Response policies -> Client

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 12 - Azure PaaS, Serverless, Logic Apps, Azure SQL, and Cosmos DB

App Service and Deployment

- App Service hosts web apps and APIs without managing servers. The App Service Plan controls compute, scaling, and cost.
- Deployment slots support staging, warm-up, swap, and rollback. Use them for safer production releases.
- Application configuration should come from App Service settings, Key Vault references, or environment variables.

Typical deployment checks:
Build succeeds
Connection string configured
App settings configured
Database migration handled
Logs enabled

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Azure Functions

- Functions are serverless event-driven units. Common triggers include HTTP, timer, queue, blob, Event Grid, and Service Bus.
- Bindings simplify input/output integration. Keep functions focused, idempotent, and observable.
- Function keys protect HTTP functions but are not a complete identity system. Use stronger auth for production APIs.
- Durable Functions support orchestrated long-running workflows.

```
[Function("Hello")]
public IActionResult Run([HttpTrigger(AuthorizationLevel.Function, "get")] HttpRequest req)
{
    return new OkObjectResult("Hello");
}
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Logic Apps and Workflow Automation

- Logic Apps automate workflows using connectors, triggers, and actions. They are useful for integration, approvals, scheduled jobs, and low-code process automation.
- Single-tenant Logic Apps run in a dedicated environment and offer more isolation. Multi-tenant Logic Apps use shared infrastructure.
- Automated deployment should use templates or infrastructure as code so workflows can move across environments.

Examples:
New file in Blob -> send approval email -> write result to database
Schedule -> call API -> post Teams message

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Azure SQL and Cosmos DB

- Azure SQL is relational and suits structured data, transactions, joins, stored procedures, and familiar SQL Server tooling.
- Managed Instance offers broader SQL Server compatibility. Elastic pools share resources across databases.
- Cosmos DB is NoSQL, globally distributed, partitioned, and RU-based. Choose partition keys carefully because they decide scale and cost.
- Cosmos DB consistency levels include strong, bounded staleness, session, consistent prefix, and eventual. Each trades freshness, latency, and availability.

Cosmos design rule:
Pick a partition key with high cardinality, even distribution, and common query alignment.

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Part 13 - Git, Sprint Implementation, OCEAN L1, Gen AI, and Power Skills

Git Workflow

- Git tracks project history through commits. The daily loop is status, add, commit, pull, push.
- Branches isolate features. Pull requests enable review. Merge conflicts must be resolved by understanding both sides.
- Rewrite history carefully. Rebasing and reset are powerful but can confuse shared branches if used carelessly.

```
git status
git switch -c feature/library-api
git add .
git commit -m "Add library API endpoints"
git push -u origin feature/library-api
```

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Sprint Implementation Readiness

- A Sprint project should have clear requirements, domain model, API contract, database design, UI flow, authentication needs, deployment plan, and test strategy.
- Start with a small vertical slice: one entity, one database table, one API endpoint, one UI screen, one test. Then expand.
- Keep a daily implementation log: completed work, blockers, next tasks, and risks. This helps stand-ups and evaluations.

Vertical slice example:
Book model -> DbContext -> migration -> GET /api/books -> MVC index view -> unit test

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

OCEAN L1 and Gen AI Prep

- For OCEAN-style tests, revise definitions, scenario questions, code output, SQL output, cloud service selection, and Git command behavior.
- For Gen AI, learn prompt clarity, context, constraints, examples, verification, privacy, and responsible use. Never paste secrets into AI tools.
- Use AI as a study partner: ask for quizzes, explain errors, generate practice tasks, and compare your answer with a model answer.

Good prompt pattern:
Context -> Task -> Constraints -> Expected output -> Example -> Verification request

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Power Skills

- Communication skills include clarity, listening, questioning, email etiquette, grammar, and audience awareness.
- Presentation skills need objective, structure, timing, confident delivery, and clean visual support.
- Ownership means acknowledging work, risks, mistakes, and follow-up. Teamwork means communicating early and supporting shared goals.
- Use STAR for interviews: Situation, Task, Action, Result. Keep examples honest and measurable.

Email structure:
Subject
Greeting
Purpose
Context
Action needed
Deadline
Closing

Revision checkpoint: define the topic, give one real project use case, name one common mistake, and write a minimal example from memory.

Related Topics Beyond the Syllabus

These topics are strongly related to the Sprint programme and should be revised even where they are not explicitly listed as a separate syllabus day.

Clean Architecture	Separate domain logic, application services, infrastructure, and presentation so the project remains testable and maintainable.
SOLID Principles	Single responsibility, open/closed, Liskov substitution, interface segregation, and dependency inversion support better class design.
Docker	Know images, containers, ports, volumes, Dockerfile, docker compose, and why containers help local SQL/API setups.
CI/CD	Automate restore, build, test, publish, deployment, environment variables, and rollback.
Observability	Use logs, metrics, traces, correlation ids, health checks, and Application Insights.
Security Hardening	Use HTTPS, secure secrets, least privilege, input validation, upload validation, rate limiting, and dependency updates.
Transactions and Concurrency	Use database transactions for multi-step updates and concurrency tokens for simultaneous edits.
API Documentation	Use Swagger descriptions, examples, response codes, and versioning policies.
Accessibility	Use labels, keyboard navigation, alt text, focus indicators, and sufficient color contrast.
Performance	Watch N+1 queries, missing indexes, unnecessary tracking, large payloads, blocking calls, and unbounded pagination.

Syllabus Focus Sheet 1: Introduction to .NET Core

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Introduction to .NET Core
- .NET Core – Overview
- .NET Platform Overview
- Characteristics of .NET Core
- Tooling

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 2: .NET Core Architecture & Platform

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- .NET Core Architecture & Platform
- The .NET Core Platform
- .NET CORE architecture and Advantages
- Build and run Cross platform apps
- New Features in .NET 8

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 3: .NET Core Setup & Execution

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- .NET Core Setup & Execution
- .NET Core – Environment Setup
- .NET Core – Code Execution
- .NET Core – Modularity
- .NET Core – Project Files

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 4: Middleware & Deployment

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Middleware & Deployment
- Middleware
- IIS Publishing
- Different cross platform deployments
- .NET Core – Windows Runtime and Extension SDKs

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Middleware is ordered. A request passes through middleware in registration order, and the response returns in reverse order. Misordering UseAuthentication and UseAuthorization can break security.
- Deployment is not only publishing files. You must decide runtime, hosting model, environment configuration, database connectivity, logging, and rollback approach.
- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 5: Libraries, Framework Comparison & Microservices

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Libraries, Framework Comparison & Microservices
- .NET Core – Create .NET Standard Library
- What is .NET Framework
- Comparison between .NET Framework & .NET Core
- Microservices using .NET Core
- Introduction
- Key benefits – Scalability, Resilience, Independence

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Microservices split a system into independently deployable services. Benefits include team autonomy and scaling, but costs include distributed tracing, data consistency, network failures, and deployment complexity.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 6: Introduction to C

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Introduction to C
- Features of C
- C# Compilation and Execution
- General Structure of a C# Program

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 7: Introduction to Power skills

Topic family: Power Skill 1

Concepts to Cover

- Introduction to Power skills
- Ice breaking, Objective setting
- Communication skills (Modes and overview)
- Introduction to written communication (dos and don'ts of emails)
- Grooming (Hygiene Squad)

Detailed Notes

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.
- Practice STAR stories for behavioral interviews: Situation, Task, Action, Result. Use real training examples such as debugging, teamwork, deadlines, and learning a new tool.

Important Details

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.

How to Practice

- Prepare one written email, one 2-minute presentation, and one STAR story from your training.

Interview Angle

- Expect email etiquette, presentation structure, teamwork, ownership, and behavioral examples.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 8: Data Types in C

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Data Types in C
- Value Types and Reference Types
- Boxing and Unboxing
- Nullable Types
- Implicitly Typed Local Variables
- Var vs Dynamic

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Boxing copies a value type into an object reference; unboxing extracts it back. It adds allocation and casting risk, so generics are usually preferred.
- Nullable value types such as `int?` represent a value or no value. Nullable reference types help the compiler warn about possible null references.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 9: Arrays & Core Language Features

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Arrays & Core Language Features
- Data Types and Arrays in C
- Single Dimensional Arrays
- Multi-Dimensional Arrays
- Jagged Arrays
- Is and As Operator
- Ref vs Out Keywords

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Arrays are fixed-size indexed collections. Use them when size is known; use List<T> when the collection grows dynamically.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 10: Core Object & String Handling

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Core Object & String Handling
- The 'object' base class in .NET
- Equals() vs ==
- String vs StringBuilder
- String Manipulation
- Various String Class Methods
- Default Parameters
- Named Parameters

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- String is immutable, so repeated concatenation creates new strings. StringBuilder is better for many incremental modifications.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 11: Parsing & Debugging

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Parsing & Debugging
- Parse() vs TryParse() vs Convert Class Methods
- Debugging in C
- Various Types of .NET Projects
- Tracing, Debugging, Build
- Compile Options

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Parse throws when conversion fails, TryParse returns a bool and is safer for user input, and Convert handles nulls differently for some types.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 12: Debugging Tools & Practices

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Debugging Tools & Practices
- Using Breakpoints
- Using Break Conditions
- Using Watch and Output Window
- Creating Multiple Projects within One Solution
- Customizing Visual Studio Settings
- Extensions
- NuGet Packages
- Environment Settings

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- A breakpoint pauses execution so you can inspect variables, call stack, and control flow. Conditional breakpoints are excellent for loops and rare cases.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 13: Logging & Performance Diagnostics

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Logging & Performance Diagnostics
- Common Debugging Practices
- Using Logs and Trace Statements
- Structured Logging (Serilog, NLog)
- Diagnosing Performance Bottlenecks
- Profilers
- Performance Counters

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Structured logging records named fields instead of only plain text. This makes logs searchable by request id, user id, endpoint, and error type.
- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 14: OOP Fundamentals

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- OOP Fundamentals
- OOP with C
- Structures and Enums
- Architecture of a Class
- Instance, Class & Reference Variables
- Access Modifiers

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 15: Inheritance & Polymorphism

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Inheritance & Polymorphism
- Abstract Classes
- Constructors and Destructors
- .NET Base Class Library
- Inheritance in C
- Polymorphism
- Method Overloading
- Method Overriding

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Inheritance models an is-a relationship. Use it for true specialization, not just code reuse. Prefer composition when behavior varies independently.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 16: Advanced OOP Concepts

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Advanced OOP Concepts
- Operator Overloading
- Method Hiding
- Access Modifiers
- Private, Public, Protected, Internal, Protected Internal, New
- Encapsulation
- Abstraction
- Sealed Classes

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 17: Interfaces, Generics & Language Enhancements

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Interfaces, Generics & Language Enhancements
- Creating Interfaces
- Implementing Interface Inheritance
- Declaring Properties within Interfaces
- Namespaces
- Creating and Using Generic Classes
- Indexers & Properties
- Auto-Implemented Properties
- Static Classes, Methods & Members

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Inheritance models an is-a relationship. Use it for true specialization, not just code reuse. Prefer composition when behavior varies independently.
- Interfaces define contracts. They support dependency injection, testing with mocks, and interchangeable implementations.
- Generics let one type or method work with many data types while preserving compile-time type safety.
- Indexes speed reads but add write cost and storage. Index columns used frequently in search, joins, filters, and ordering.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 18: Advanced Language Features & Regex

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Advanced Language Features & Regex
- Property Accessors
- Partial Types
- Extension Methods
- Object Initializer
- Anonymous Types
- Evaluating Regular Expressions
- RegEx Class
- Forming Regular Expressions
- RegEx Methods

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Regular expressions are powerful but can become unreadable. Anchor patterns, keep them simple, and test valid plus invalid examples.
- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 19: Exception Handling

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Exception Handling
- Exception Class Hierarchy
- Try Block
- Multiple Catch Blocks
- Finally Block
- Throwing Exceptions
- Throw Keyword
- Inner Exception
- Custom Exceptions
- Best Practices
- Garbage Collection
- Finalize vs Dispose

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Catch exceptions only when you can add value, recover, translate, or log. Do not swallow errors silently.
- Garbage collection reclaims managed memory. Dispose is still needed for unmanaged resources such as files, streams, database connections, and sockets.
- Locks protect shared mutable data. Keep critical sections short and avoid locking on public objects.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 20: Collections, Delegates & File Handling

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Collections, Delegates & File Handling
- System.Collections Namespace
- Collection Interfaces & Classes
- HashSet, Queue, Stack, LinkedList
- Collection API
- Generics
- Delegates & Events
- Anonymous Methods
- Lambda Expressions
- Expression Trees
- File I/O
- Serialization (SOAP, XML, JSON)

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Interfaces define contracts. They support dependency injection, testing with mocks, and interchangeable implementations.
- Generics let one type or method work with many data types while preserving compile-time type safety.
- Delegates represent method references. Events build on delegates to implement publish-subscribe behavior.
- Lambda expressions are concise anonymous functions, heavily used in LINQ, events, and callbacks.
- Queues decouple producers and consumers. They improve resilience when downstream processing is slow or temporarily unavailable.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 21: Threading, Parallel & Async Programming

Topic family: .NET 8.0 and C# 12.0

Concepts to Cover

- Threading, Parallel & Async Programming
- Task Parallel Library
- Threads vs Tasks
- Task-Based Asynchronous Model
- Async and Await
- Synchronization & Locks
- Features in C# 8.0
- Features in C# 10.0
- New Features in C# 12
- Experimental Attributes
- Interceptors

Detailed Notes

- .NET is a managed platform built around a runtime, base class libraries, SDK tooling, and language compilers. In Sprint work, think in terms of project file, Program.cs, dependencies, build output, and runtime behavior.
- C# code should be read by separating syntax from intent: what data type is being modeled, what lifetime it has, how errors are handled, and whether the code is synchronous, asynchronous, mutable, or immutable.
- A strong answer connects language features to maintainability. For example, interfaces support loose coupling, generics avoid repeated casting, async improves scalability for I/O, and exceptions create a standard error flow.

Important Details

- Async/await is best for I/O-bound work. It does not automatically make CPU-bound work faster, but it prevents blocking request threads.
- Locks protect shared mutable data. Keep critical sections short and avoid locking on public objects.

How to Practice

- Create a small console app that demonstrates the concept, then explain the output line by line.

Interview Angle

- Be ready to explain what problem the feature solves, not only the syntax.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 22: Introduction to Data Structures

Topic family: Programming Foundation and Problem Solving

Concepts to Cover

- Introduction to Data Structures
- Primitive Data Structures
- Arrays
- Strings Non-Primitive Data Structures (Linear)
- Non-Primitive Data Structures – Overview
- Stacks
- Queues
- Lists

Detailed Notes

- Problem solving is not only memorizing algorithms. It is the habit of identifying input, output, constraints, edge cases, state changes, and complexity before writing code.
- Data structures matter because each one optimizes a different operation. Arrays offer indexed access, stacks model last-in-first-out, queues model first-in-first-out, trees model hierarchy, and graphs model relationships.
- In assessments, always discuss time complexity and space complexity. A correct $O(n \log n)$ sort may be preferable to a simple $O(n^2)$ sort when input size grows.

Important Details

- Arrays are fixed-size indexed collections. Use them when size is known; use `List<T>` when the collection grows dynamically.
- Queues decouple producers and consumers. They improve resilience when downstream processing is slow or temporarily unavailable.

How to Practice

- Solve one problem by writing input, output, edge cases, pseudocode, code, and Big-O analysis.

Interview Angle

- Always mention edge cases and complexity before declaring a solution final.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 23: Non-Primitive Data Structures (Non-Linear)

Topic family: Programming Foundation and Problem Solving

Concepts to Cover

- Non-Primitive Data Structures (Non-Linear)
- Trees
- Graphs

Detailed Notes

- Problem solving is not only memorizing algorithms. It is the habit of identifying input, output, constraints, edge cases, state changes, and complexity before writing code.
- Data structures matter because each one optimizes a different operation. Arrays offer indexed access, stacks model last-in-first-out, queues model first-in-first-out, trees model hierarchy, and graphs model relationships.
- In assessments, always discuss time complexity and space complexity. A correct $O(n \log n)$ sort may be preferable to a simple $O(n^2)$ sort when input size grows.

Important Details

- Problem solving is not only memorizing algorithms. It is the habit of identifying input, output, constraints, edge cases, state changes, and complexity before writing code.
- Data structures matter because each one optimizes a different operation. Arrays offer indexed access, stacks model last-in-first-out, queues model first-in-first-out, trees model hierarchy, and graphs model relationships.

How to Practice

- Solve one problem by writing input, output, edge cases, pseudocode, code, and Big-O analysis.

Interview Angle

- Always mention edge cases and complexity before declaring a solution final.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 24: Algorithm Analysis & Searching

Topic family: Programming Foundation and Problem Solving

Concepts to Cover

- Algorithm Analysis & Searching
- Analysis of Algorithms
- Linear Search
- Binary Search
- Analysis of Searching Algorithms

Detailed Notes

- Problem solving is not only memorizing algorithms. It is the habit of identifying input, output, constraints, edge cases, state changes, and complexity before writing code.
- Data structures matter because each one optimizes a different operation. Arrays offer indexed access, stacks model last-in-first-out, queues model first-in-first-out, trees model hierarchy, and graphs model relationships.
- In assessments, always discuss time complexity and space complexity. A correct $O(n \log n)$ sort may be preferable to a simple $O(n^2)$ sort when input size grows.

Important Details

- Linear search checks each item and is $O(n)$. It works on unsorted data but becomes slow as input grows.
- Binary search is $O(\log n)$, but it requires sorted data and careful boundary handling.

How to Practice

- Solve one problem by writing input, output, edge cases, pseudocode, code, and Big-O analysis.

Interview Angle

- Always mention edge cases and complexity before declaring a solution final.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 25: Basic Sorting Algorithms

Topic family: Programming Foundation and Problem Solving

Concepts to Cover

- Basic Sorting Algorithms
- Bubble Sort
- Selection Sort
- Insertion Sort
- Analysis of Sorting Algorithms Advanced Sorting Algorithms
- Shell Sort
- Merge Sort
- Quick Sort
- Comparative Analysis of Sorting Algorithms

Detailed Notes

- Problem solving is not only memorizing algorithms. It is the habit of identifying input, output, constraints, edge cases, state changes, and complexity before writing code.
- Data structures matter because each one optimizes a different operation. Arrays offer indexed access, stacks model last-in-first-out, queues model first-in-first-out, trees model hierarchy, and graphs model relationships.
- In assessments, always discuss time complexity and space complexity. A correct $O(n \log n)$ sort may be preferable to a simple $O(n^2)$ sort when input size grows.

Important Details

- Merge sort is stable and $O(n \log n)$, but needs extra memory for merging.
- Quick sort is fast on average but can degrade to $O(n^2)$ without good pivot strategy.

How to Practice

- Solve one problem by writing input, output, edge cases, pseudocode, code, and Big-O analysis.

Interview Angle

- Always mention edge cases and complexity before declaring a solution final.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 26: Language skills (Part1)

Topic family: Power Skill 2

Concepts to Cover

- Language skills (Part1)
- Introduction to parts of speech & Subject verb agreement and Articles
- Content videos
- Email Etiquette
- Test(Email- application of SVA & Articles).
- Activity bases learnings (role play activity)
- Listening (questioning and probing)
- Language skills (Part2)
- Tenses
- Art of pronunciation (verbal's skills)
- using Co-pilot and ChatGPT

Detailed Notes

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.
- Practice STAR stories for behavioral interviews: Situation, Task, Action, Result. Use real training examples such as debugging, teamwork, deadlines, and learning a new tool.

Important Details

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.

How to Practice

- Prepare one written email, one 2-minute presentation, and one STAR story from your training.

Interview Angle

- Expect email etiquette, presentation structure, teamwork, ownership, and behavioral examples.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 27: Recursion

Topic family: Programming Foundation and Problem Solving

Concepts to Cover

- Recursion
- Introduction to Recursion
- Tail Recursion
- Head Recursion
- Tree Recursion
- Indirect Recursion

Detailed Notes

- Problem solving is not only memorizing algorithms. It is the habit of identifying input, output, constraints, edge cases, state changes, and complexity before writing code.
- Data structures matter because each one optimizes a different operation. Arrays offer indexed access, stacks model last-in-first-out, queues model first-in-first-out, trees model hierarchy, and graphs model relationships.
- In assessments, always discuss time complexity and space complexity. A correct $O(n \log n)$ sort may be preferable to a simple $O(n^2)$ sort when input size grows.

Important Details

- Every recursive solution needs a base case and progress toward it. Also consider stack depth.

How to Practice

- Solve one problem by writing input, output, edge cases, pseudocode, code, and Big-O analysis.

Interview Angle

- Always mention edge cases and complexity before declaring a solution final.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 28: Installation and Setup

Topic family: Unit Testing Framework (XUNIT, NUnit)

Concepts to Cover

- Installation and Setup
- Writing Test Methods
- Assertions
- Test Lifecycle and Hooks
- Test Execution
- Parameterized Tests
- Test Organization

Detailed Notes

- Unit tests validate small units of behavior in isolation. A useful test has clear arrange, act, and assert phases, and it should fail for one understandable reason.
- xUnit and NUnit both support facts/tests, setup/teardown, parameterized data, assertions, fixtures, and parallel execution. Know the vocabulary even if you use one framework more often.
- Mocking is used when a class depends on external systems such as databases, APIs, queues, or file systems. The test should verify business behavior, not re-test the framework.

Important Details

- Assertions should express expected behavior clearly. Prefer specific assertions over vague true/false checks when possible.

How to Practice

- Write three tests: success case, validation failure, and exception/edge case.

Interview Angle

- Explain the difference between unit, integration, and end-to-end testing.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 29: Skipping and Ignoring Tests

Topic family: Unit Testing Framework (XUNIT, NUnit)

Concepts to Cover

- Skipping and Ignoring Tests
- Test Output and Logging
- Test Fixtures and Shared Context
- Data-Driven Tests
- Mocking and Stubbing
- Exception Handling
- Parallel Test Execution

Detailed Notes

- Unit tests validate small units of behavior in isolation. A useful test has clear arrange, act, and assert phases, and it should fail for one understandable reason.
- xUnit and NUnit both support facts/tests, setup/teardown, parameterized data, assertions, fixtures, and parallel execution. Know the vocabulary even if you use one framework more often.
- Mocking is used when a class depends on external systems such as databases, APIs, queues, or file systems. The test should verify business behavior, not re-test the framework.

Important Details

- Catch exceptions only when you can add value, recover, translate, or log. Do not swallow errors silently.
- Mock external dependencies such as repositories, email senders, queues, and HTTP clients. Do not mock simple value objects.

How to Practice

- Write three tests: success case, validation failure, and exception/edge case.

Interview Angle

- Explain the difference between unit, integration, and end-to-end testing.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 30: Introduction to Databases – SQL & NoSQL

Topic family: RDBMS & SQL Server

Concepts to Cover

- Introduction to Databases – SQL & NoSQL
- Introduction to RDBMS
- Data Models in Database
- Properties of RDBMS
- Codd's Relational Database Rules
- Data Integrity
- Normalization T-SQL Language
- Beginning with Transact-SQL
- Working with Data Types (Basics)
- Working with Schema
- Working with Tables
- DDL, DML, DCL Statements
- Implementing Data Integrity

Detailed Notes

- RDBMS thinking starts with tables, rows, columns, keys, constraints, and relationships. Good schema design prevents invalid data before application code even runs.
- SQL Server work often combines DDL for schema, DML for data changes, DCL for permissions, joins for related data, indexes for performance, and stored procedures for reusable database logic.
- Normalize to remove duplication, then denormalize only when there is a proven reporting or performance reason. Always understand primary keys, foreign keys, unique constraints, and nullability.

Important Details

- Normalization reduces duplicate data and update anomalies. Know 1NF, 2NF, and 3NF at least.

How to Practice

- Create a tiny schema, insert sample rows, and write one select, join, aggregate, and stored procedure.

Interview Angle

- Expect questions on keys, normalization, joins, indexes, and stored procedures.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 31: Transact-SQL & System Functions

Topic family: RDBMS & SQL Server

Concepts to Cover

- Transact-SQL & System Functions
- Transact-SQL
- System Functions
- Advanced T-SQL Queries
- Advanced T-SQL Statements
- Other T-SQL Statements
- Set Operators

Detailed Notes

- RDBMS thinking starts with tables, rows, columns, keys, constraints, and relationships. Good schema design prevents invalid data before application code even runs.
- SQL Server work often combines DDL for schema, DML for data changes, DCL for permissions, joins for related data, indexes for performance, and stored procedures for reusable database logic.
- Normalize to remove duplication, then denormalize only when there is a proven reporting or performance reason. Always understand primary keys, foreign keys, unique constraints, and nullability.

Important Details

- Functions are event-driven and scale based on triggers. Keep functions focused and idempotent where possible.

How to Practice

- Create a tiny schema, insert sample rows, and write one select, join, aggregate, and stored procedure.

Interview Angle

- Expect questions on keys, normalization, joins, indexes, and stored procedures.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 32: Joins & Subqueries

Topic family: RDBMS & SQL Server

Concepts to Cover

- Joins & Subqueries
- What are Joins?
- Types of Joins
- Working with Joins
- Subqueries

Detailed Notes

- RDBMS thinking starts with tables, rows, columns, keys, constraints, and relationships. Good schema design prevents invalid data before application code even runs.
- SQL Server work often combines DDL for schema, DML for data changes, DCL for permissions, joins for related data, indexes for performance, and stored procedures for reusable database logic.
- Normalize to remove duplication, then denormalize only when there is a proven reporting or performance reason. Always understand primary keys, foreign keys, unique constraints, and nullability.

Important Details

- Joins combine rows across related tables. Inner joins require matches; left joins keep unmatched rows from the left table.

How to Practice

- Create a tiny schema, insert sample rows, and write one select, join, aggregate, and stored procedure.

Interview Angle

- Expect questions on keys, normalization, joins, indexes, and stored procedures.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 33: Views & Indexes

Topic family: RDBMS & SQL Server

Concepts to Cover

- Views & Indexes
- Introduction to Views
- Introduction to Indexes Stored Procedures
- Implementing Stored Procedures with Input Parameters
- Implementing Stored Procedures with Output Parameters Exception Handling using TRY-CATCH
- Stored Procedure Debugging
- Execution Plan

Detailed Notes

- RDBMS thinking starts with tables, rows, columns, keys, constraints, and relationships. Good schema design prevents invalid data before application code even runs.
- SQL Server work often combines DDL for schema, DML for data changes, DCL for permissions, joins for related data, indexes for performance, and stored procedures for reusable database logic.
- Normalize to remove duplication, then denormalize only when there is a proven reporting or performance reason. Always understand primary keys, foreign keys, unique constraints, and nullability.

Important Details

- Catch exceptions only when you can add value, recover, translate, or log. Do not swallow errors silently.
- Indexes speed reads but add write cost and storage. Index columns used frequently in search, joins, filters, and ordering.
- Stored procedures centralize database logic and can simplify permissions, but too much business logic in SQL can become hard to test.

How to Practice

- Create a tiny schema, insert sample rows, and write one select, join, aggregate, and stored procedure.

Interview Angle

- Expect questions on keys, normalization, joins, indexes, and stored procedures.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 34: LINQ Fundamentals & Syntax

Topic family: LINQ

Concepts to Cover

- LINQ Fundamentals & Syntax
- Language Integrated Query – Introduction
- LINQ Syntax
- Query Syntax vs Method Syntax
- Introduction to System.Linq.Queryable
- LINQ to Objects
- LINQ to Object Core LINQ Operators
- Query Operators
- from, select, where
- ofType
- OrderBy, ThenBy
- GroupBy, into
- Select, SelectMany
- Take, TakeWhile
- First, FirstOrDefault
- Single, SingleOrDefault

Detailed Notes

- LINQ gives a consistent query style over objects, EF Core queries, XML, and other providers. The same-looking LINQ expression can execute in memory or be translated to SQL.
- Query syntax resembles SQL, while method syntax chains extension methods. You should be comfortable reading both because projects often mix them.
- Deferred execution means a query is not executed until enumerated. Immediate execution happens with operators such as ToList, Count, First, Single, Sum, and Average.

Important Details

- LINQ gives a consistent query style over objects, EF Core queries, XML, and other providers. The same-looking LINQ expression can execute in memory or be translated to SQL.
- Query syntax resembles SQL, while method syntax chains extension methods. You should be comfortable reading both because projects often mix them.

How to Practice

- Write the same query using query syntax and method syntax, then explain when it executes.

Interview Angle

- Expect deferred execution, IQueryable vs IEnumerable, and First vs Single.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 35: Aggregate Functions

Topic family: LINQ

Concepts to Cover

- Aggregate Functions
- Sum, Min, Max, Average, Count
- Distinct
- Intersect
- Except
- Join
- LINQ Projection
- Deferred Execution vs Immediate Execution
- Let Keyword
- LINQ to DataTable

Detailed Notes

- LINQ gives a consistent query style over objects, EF Core queries, XML, and other providers. The same-looking LINQ expression can execute in memory or be translated to SQL.
- Query syntax resembles SQL, while method syntax chains extension methods. You should be comfortable reading both because projects often mix them.
- Deferred execution means a query is not executed until enumerated. Immediate execution happens with operators such as ToList, Count, First, Single, Sum, and Average.

Important Details

- Joins combine rows across related tables. Inner joins require matches; left joins keep unmatched rows from the left table.
- Deferred LINQ queries execute when enumerated. If source data changes before enumeration, results may change too.
- Functions are event-driven and scale based on triggers. Keep functions focused and idempotent where possible.

How to Practice

- Write the same query using query syntax and method syntax, then explain when it executes.

Interview Angle

- Expect deferred execution, IQueryable vs IEnumerable, and First vs Single.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 36: ORM & EF Core Overview

Topic family: Entity Framework Core

Concepts to Cover

- ORM & EF Core Overview
- Entity Framework Core
- Overview of ORM Products
- Entity Framework Introduction

Detailed Notes

- EF Core is an ORM that maps C# classes to database tables and tracks changes through DbContext. It reduces boilerplate but does not remove the need to understand SQL.
- Code First begins with entities and migrations. Database First begins with an existing schema and scaffolds classes. Both still require relationship and configuration awareness.
- Loading strategy matters. Eager loading uses Include, explicit loading loads related data when requested, and lazy loading loads on navigation access but can cause hidden N+1 queries.

Important Details

- EF Core is an ORM that maps C# classes to database tables and tracks changes through DbContext. It reduces boilerplate but does not remove the need to understand SQL.
- Code First begins with entities and migrations. Database First begins with an existing schema and scaffolds classes. Both still require relationship and configuration awareness.

How to Practice

- Create two related entities, add a migration, seed data, query with Include, and update one record.

Interview Angle

- Expect DbContext, migrations, tracking, Include, Code First vs DB First, and repository pattern.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 37: EF Core Approaches

Topic family: Entity Framework Core

Concepts to Cover

- EF Core Approaches
- Using Database First Approach
- Using Code First Approach
- Setting up Entities in EF Core Repository Pattern & CRUD
- Implementing Repository Pattern
- Repository Pattern
- Introduction
- Benefits
- Repository Pattern Implementation
- Using LINQ to Entities to Perform CRUD Operations
- SQL Query Logging

Detailed Notes

- LINQ gives a consistent query style over objects, EF Core queries, XML, and other providers. The same-looking LINQ expression can execute in memory or be translated to SQL.
- Query syntax resembles SQL, while method syntax chains extension methods. You should be comfortable reading both because projects often mix them.
- Deferred execution means a query is not executed until enumerated. Immediate execution happens with operators such as ToList, Count, First, Single, Sum, and Average.

Important Details

- Repository pattern hides data access details behind an interface. It is useful when it simplifies testing and isolates persistence logic.
- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Write the same query using query syntax and method syntax, then explain when it executes.

Interview Angle

- Expect deferred execution, IQueryable vs IEnumerable, and First vs Single.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 38: Migrations & Advanced Data Access

Topic family: Entity Framework Core

Concepts to Cover

- Migrations & Advanced Data Access
- Migration & Database Update
- Eager Loading vs Explicit Loading vs Lazy Loading
- Raw SQL
- Stored Procedures

Detailed Notes

- EF Core is an ORM that maps C# classes to database tables and tracks changes through DbContext. It reduces boilerplate but does not remove the need to understand SQL.
- Code First begins with entities and migrations. Database First begins with an existing schema and scaffolds classes. Both still require relationship and configuration awareness.
- Loading strategy matters. Eager loading uses Include, explicit loading loads related data when requested, and lazy loading loads on navigation access but can cause hidden N+1 queries.

Important Details

- Stored procedures centralize database logic and can simplify permissions, but too much business logic in SQL can become hard to test.
- Migrations are versioned schema changes. Always review generated migrations before applying them.
- Eager loading with Include prevents lazy N+1 issues when you already know related data is needed.
- Raw SQL is useful for complex queries, but parameterize inputs to avoid injection.

How to Practice

- Create two related entities, add a migration, seed data, query with Include, and update one record.

Interview Angle

- Expect DbContext, migrations, tracking, Include, Code First vs DB First, and repository pattern.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 39: HTML5

Topic family: Web Basics (HTML5, CSS 3, JavaScript)

Concepts to Cover

- HTML5
- HTML – Introduction
- HTML Elements and Structure
- HTML Basic Formatting Tags
- HTML Headers
- HTML Grouping Using Div and Span HTML Lists
- HTML Images
- HTML Hyperlink
- HTML Table
- HTML Form
- <form> tag, Method: GET vs POST, <input> types (text, password, email)
- Labels, Placeholders, Fieldsets, Legends
- Client-side validation (HTML5 validation attributes)

Detailed Notes

- The web stack has layers: HTML for structure, CSS for presentation, JavaScript for behavior, HTTP for communication, and browser APIs for storage and events.
- Forms are central to MVC and API work. Understand input names, GET versus POST, labels, placeholders, validation attributes, and how submitted values become server-side model properties.
- Client-side validation helps users, but server-side validation protects the application. Never trust browser checks alone.

Important Details

- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.

How to Practice

- Build one responsive form and validate it with HTML attributes, JavaScript, and server-side rules.

Interview Angle

- Expect GET vs POST, semantic HTML, CSS box model, DOM events, and validation.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 40: Semantic HTML & Client Storage

Topic family: Web Basics (HTML5, CSS 3, JavaScript)

Concepts to Cover

- Semantic HTML & Client Storage
- Semantic HTML
- HTML Client Storage CSS Fundamentals
- CSS Introduction
- CSS Syntax
- CSS Selectors
- Color Background Cursor
- Text Fonts

Detailed Notes

- The web stack has layers: HTML for structure, CSS for presentation, JavaScript for behavior, HTTP for communication, and browser APIs for storage and events.
- Forms are central to MVC and API work. Understand input names, GET versus POST, labels, placeholders, validation attributes, and how submitted values become server-side model properties.
- Client-side validation helps users, but server-side validation protects the application. Never trust browser checks alone.

Important Details

- Semantic HTML improves accessibility, SEO, and maintainability. Use header, nav, main, section, article, aside, and footer where appropriate.

How to Practice

- Build one responsive form and validate it with HTML attributes, JavaScript, and server-side rules.

Interview Angle

- Expect GET vs POST, semantic HTML, CSS box model, DOM events, and validation.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 41: CSS Layout & Positioning

Topic family: Web Basics (HTML5, CSS 3, JavaScript)

Concepts to Cover

- CSS Layout & Positioning
- Box Model (content, padding, border, margin)
- Positioning (Static, Relative, Absolute, Fixed, Sticky)
- Display Properties (Block, Inline, Inline-block)
- Flex
- Grid basics
- CSS Floats

Detailed Notes

- The web stack has layers: HTML for structure, CSS for presentation, JavaScript for behavior, HTTP for communication, and browser APIs for storage and events.
- Forms are central to MVC and API work. Understand input names, GET versus POST, labels, placeholders, validation attributes, and how submitted values become server-side model properties.
- Client-side validation helps users, but server-side validation protects the application. Never trust browser checks alone.

Important Details

- Locks protect shared mutable data. Keep critical sections short and avoid locking on public objects.
- Flexbox is one-dimensional layout for rows or columns. Grid is two-dimensional layout for rows and columns together.

How to Practice

- Build one responsive form and validate it with HTML attributes, JavaScript, and server-side rules.

Interview Angle

- Expect GET vs POST, semantic HTML, CSS box model, DOM events, and validation.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 42: Iterators

Topic family: Web Basics (HTML5, CSS 3, JavaScript)

Concepts to Cover

- Iterators
- Generators
- DOM Manipulation
- Event Handling

Detailed Notes

- The web stack has layers: HTML for structure, CSS for presentation, JavaScript for behavior, HTTP for communication, and browser APIs for storage and events.
- Forms are central to MVC and API work. Understand input names, GET versus POST, labels, placeholders, validation attributes, and how submitted values become server-side model properties.
- Client-side validation helps users, but server-side validation protects the application. Never trust browser checks alone.

Important Details

- DOM manipulation changes the live document. Use event listeners and avoid mixing too much HTML string construction with logic.

How to Practice

- Build one responsive form and validate it with HTML attributes, JavaScript, and server-side rules.

Interview Angle

- Expect GET vs POST, semantic HTML, CSS box model, DOM events, and validation.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 43: ASP.NET Core Introduction

Topic family: ASP.NET Core MVC

Concepts to Cover

- ASP.NET Core Introduction
- .NET Core Theory
- Introduction to ASP.NET Core
- Setting ASP.NET Core Development Environment
- Setup and Project Overview
- Creating an ASP.NET Core Project
- Project File and Program File .NET Core Pipeline and Middleware
- Routing in MVC and Endpoints
- Dependency Injection
- Understanding DI
- Dependency Chains & Dependency Methods

Detailed Notes

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.
- Middleware forms the request pipeline. Routing selects endpoints, static files serve assets, authentication identifies users, authorization checks permissions, and endpoints execute actions.

Important Details

- Middleware is ordered. A request passes through middleware in registration order, and the response returns in reverse order. Misordering UseAuthentication and UseAuthorization can break security.
- DI provides dependencies through constructors or services. It reduces tight coupling and supports testing.

How to Practice

- Create an MVC CRUD screen with view model, validation, layout, and a partial view.

Interview Angle

- Expect MVC flow, routing, model binding, filters, ViewBag vs ViewData vs TempData, and Razor Pages vs MVC.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 44: Presentation skills

Topic family: Power Skill 3

Concepts to Cover

- Presentation skills
- Introduction to Presentation skills (setting objective)
- Albert Mehrabian rule and communication styles
- Rules of presentation (3 T's)
- Dos and Don'ts of presentation
- Activity on presentation skills by YP's

Detailed Notes

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.
- Practice STAR stories for behavioral interviews: Situation, Task, Action, Result. Use real training examples such as debugging, teamwork, deadlines, and learning a new tool.

Important Details

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.

How to Practice

- Prepare one written email, one 2-minute presentation, and one STAR story from your training.

Interview Angle

- Expect email etiquette, presentation structure, teamwork, ownership, and behavioral examples.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 45: Controllers and Routing

Topic family: ASP.NET Core MVC

Concepts to Cover

- Controllers and Routing
- Introduction to Controller
- Creating Controller
- Introduction to Routing
- Conventional Routing
- UseEndpoints
- MapControllerRoute
- Attribute Based Routing
- Route Parameters and Optional Parameters
- Areas

Detailed Notes

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.
- Middleware forms the request pipeline. Routing selects endpoints, static files serve assets, authentication identifies users, authorization checks permissions, and endpoints execute actions.

Important Details

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.

How to Practice

- Create an MVC CRUD screen with view model, validation, layout, and a partial view.

Interview Angle

- Expect MVC flow, routing, model binding, filters, ViewBag vs ViewData vs TempData, and Razor Pages vs MVC.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 46: Controller Actions

Topic family: ASP.NET Core MVC

Concepts to Cover

- Controller Actions
- Passing Values to Actions
- Action Return Types
- Transfer Data
- Introduction to Model
- Creating Our First Model
- Code First Approach

Detailed Notes

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.
- Middleware forms the request pipeline. Routing selects endpoints, static files serve assets, authentication identifies users, authorization checks permissions, and endpoints execute actions.

Important Details

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.

How to Practice

- Create an MVC CRUD screen with view model, validation, layout, and a partial view.

Interview Angle

- Expect MVC flow, routing, model binding, filters, ViewBag vs ViewData vs TempData, and Razor Pages vs MVC.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 47: Database, Model Binding & Data Annotations

Topic family: ASP.NET Core MVC

Concepts to Cover

- Database, Model Binding & Data Annotations
- Setting Up Connection String
- Setting Up DbContext and Entity Framework
- Setting Up the DB Context
- Passing Data to the Controller
- Model Binding
- Data Annotation

Detailed Notes

- EF Core is an ORM that maps C# classes to database tables and tracks changes through DbContext. It reduces boilerplate but does not remove the need to understand SQL.
- Code First begins with entities and migrations. Database First begins with an existing schema and scaffolds classes. Both still require relationship and configuration awareness.
- Loading strategy matters. Eager loading uses Include, explicit loading loads related data when requested, and lazy loading loads on navigation access but can cause hidden N+1 queries.

Important Details

- Model binding maps HTTP data to .NET parameters or objects. Understand [FromBody], [FromForm], [FromQuery], and [FromRoute].
- Data annotations drive validation and schema hints. For complex rules, consider FluentValidation or custom validation attributes.

How to Practice

- Create two related entities, add a migration, seed data, query with Include, and update one record.

Interview Angle

- Expect DbContext, migrations, tracking, Include, Code First vs DB First, and repository pattern.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 48: ViewBag and Passing Data from Controller to View

Topic family: ASP.NET Core MVC

Concepts to Cover

- ViewBag and Passing Data from Controller to View
- ViewBag and ViewData
- ViewModels and Strongly Typed Views
- ViewModels in Action
- Layout
- Helper Class
- View Component
- Razor Pages
- PageModel Classes (OnGet, OnPost, OnPut, etc.)
- Page Handlers
- Page-Specific Routing
- Code-Behind Structure
- When to Choose Razor Pages vs MVC

Detailed Notes

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.
- Middleware forms the request pipeline. Routing selects endpoints, static files serve assets, authentication identifies users, authorization checks permissions, and endpoints execute actions.

Important Details

- View models are tailored to a screen or request. They keep UI requirements separate from database entities.
- Razor Pages group page markup with a PageModel. They suit page-focused apps; MVC suits controller/action organization and larger separation.

How to Practice

- Create an MVC CRUD screen with view model, validation, layout, and a partial view.

Interview Angle

- Expect MVC flow, routing, model binding, filters, ViewBag vs ViewData vs TempData, and Razor Pages vs MVC.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 49: Filters

Topic family: ASP.NET Core MVC

Concepts to Cover

- Filters
- Introduction to Filters
- Filter Types
- Security – Authentication
- Implementing Authentication Identity
- Token-Based Authentication (JWT)
- Authorization in .NET Core
- Deployment Modes of .NET Core Application

Detailed Notes

- ASP.NET Core MVC separates the application into Models, Views, and Controllers. Controllers receive requests, coordinate with services/data, and choose responses.
- Razor views are server-rendered templates. Strongly typed views and view models are preferred for real applications because they are clearer and safer than loose ViewBag data.
- Middleware forms the request pipeline. Routing selects endpoints, static files serve assets, authentication identifies users, authorization checks permissions, and endpoints execute actions.

Important Details

- Deployment is not only publishing files. You must decide runtime, hosting model, environment configuration, database connectivity, logging, and rollback approach.
- Filters run before or after actions and can handle authorization, resource checks, exceptions, and result processing.
- JWT bearer tokens are sent in Authorization headers. Validate issuer, audience, lifetime, and signing key.

How to Practice

- Create an MVC CRUD screen with view model, validation, layout, and a partial view.

Interview Angle

- Expect MVC flow, routing, model binding, filters, ViewBag vs ViewData vs TempData, and Razor Pages vs MVC.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 50: Introduction to .Net Core Web API

Topic family: ASP.NET Core Web API

Concepts to Cover

- Introduction to .Net Core Web API
- Introduction to Web Service
- Introduction to REST API
- Introduction to Web API
- Difference between Web Service, WCF Service and Web API
- HTTPS Verbs
- HTTP Response Status Codes

Detailed Notes

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.
- Cross-cutting concerns such as validation, error handling, logging, security, CORS, caching, and versioning should be designed deliberately instead of sprinkled randomly.

Important Details

- REST APIs use resources and standard HTTP verbs. Avoid designing every endpoint as an arbitrary action when a resource shape fits.
- Status codes are part of the API contract. Clients depend on them for success, validation, auth, and missing-resource behavior.

How to Practice

- Design REST endpoints for one resource and test them through Swagger and Postman.

Interview Angle

- Expect REST, status codes, DTOs, CORS, Swagger, authentication, logging, and versioning.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 51: Web API Routing

Topic family: ASP.NET Core Web API

Concepts to Cover

- Web API Routing
- Configuring Web API
- Startup Configuration
- API Security
- Dependency Injection in .Net Core

Detailed Notes

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.
- Cross-cutting concerns such as validation, error handling, logging, security, CORS, caching, and versioning should be designed deliberately instead of sprinkled randomly.

Important Details

- DI provides dependencies through constructors or services. It reduces tight coupling and supports testing.

How to Practice

- Design REST endpoints for one resource and test them through Swagger and Postman.

Interview Angle

- Expect REST, status codes, DTOs, CORS, Swagger, authentication, logging, and versioning.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 52: Controllers & Action Results

Topic family: ASP.NET Core Web API

Concepts to Cover

- Controllers & Action Results
- Controller Action Return Types
- Introduction to Controller Action Return Types
- Specific Type
- IActionResult
- ActionResult<Type>
- Custom Return Type

Detailed Notes

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.
- Cross-cutting concerns such as validation, error handling, logging, security, CORS, caching, and versioning should be designed deliberately instead of sprinkled randomly.

Important Details

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.

How to Practice

- Design REST endpoints for one resource and test them through Swagger and Postman.

Interview Angle

- Expect REST, status codes, DTOs, CORS, Swagger, authentication, logging, and versioning.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 53: Cross-Cutting Concerns

Topic family: ASP.NET Core Web API

Concepts to Cover

- Cross-Cutting Concerns
- Filters
- API Health Checkup
- HTTP Security Policies
- Request & Response Formatting
- Content Negotiation
- What is CORS & How to Handle in Web API

Detailed Notes

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.
- Cross-cutting concerns such as validation, error handling, logging, security, CORS, caching, and versioning should be designed deliberately instead of sprinkled randomly.

Important Details

- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.
- Filters run before or after actions and can handle authorization, resource checks, exceptions, and result processing.
- CORS is enforced by browsers. Configure only trusted origins, methods, and headers.

How to Practice

- Design REST endpoints for one resource and test them through Swagger and Postman.

Interview Angle

- Expect REST, status codes, DTOs, CORS, Swagger, authentication, logging, and versioning.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 54: Data Handling, DTOs & Validation

Topic family: ASP.NET Core Web API

Concepts to Cover

- Data Handling, DTOs & Validation
- Working with Relational Data using Entity Framework Core
- Relationships in EF Core
- DML Manipulation using Repository Pattern
- What is DTO & How to Use with AutoMapper
- Fluent Validation

Detailed Notes

- EF Core is an ORM that maps C# classes to database tables and tracks changes through DbContext. It reduces boilerplate but does not remove the need to understand SQL.
- Code First begins with entities and migrations. Database First begins with an existing schema and scaffolds classes. Both still require relationship and configuration awareness.
- Loading strategy matters. Eager loading uses Include, explicit loading loads related data when requested, and lazy loading loads on navigation access but can cause hidden N+1 queries.

Important Details

- Repository pattern hides data access details behind an interface. It is useful when it simplifies testing and isolates persistence logic.
- DTOs are especially important for Web APIs because they prevent accidental exposure of database structure.
- AutoMapper reduces repetitive mapping but can hide complexity. Manual mapping is clearer for small DTOs.
- FluentValidation keeps validation rules expressive and testable, especially when rules become conditional.

How to Practice

- Create two related entities, add a migration, seed data, query with Include, and update one record.

Interview Angle

- Expect DbContext, migrations, tracking, Include, Code First vs DB First, and repository pattern.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 55: Error Handling, Logging & Caching

Topic family: ASP.NET Core Web API

Concepts to Cover

- Error Handling, Logging & Caching
- Try–Catch–Finally Block
- Throwing Custom Exceptions
- Global Error Handling
- Custom Global Error Handling
- Web API Logging
- Caching Strategies in .Net Core Web API

Detailed Notes

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.
- Cross-cutting concerns such as validation, error handling, logging, security, CORS, caching, and versioning should be designed deliberately instead of sprinkled randomly.

Important Details

- Catch exceptions only when you can add value, recover, translate, or log. Do not swallow errors silently.
- Locks protect shared mutable data. Keep critical sections short and avoid locking on public objects.
- Global error handling creates consistent responses and avoids repeating try-catch in every action.
- Cache stable data carefully. Invalidate or expire cached values when underlying data changes.

How to Practice

- Design REST endpoints for one resource and test them through Swagger and Postman.

Interview Angle

- Expect REST, status codes, DTOs, CORS, Swagger, authentication, logging, and versioning.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 56: Versioning, Testing & API Build

Topic family: ASP.NET Core Web API

Concepts to Cover

- Versioning, Testing & API Build
- Web API Versioning
- Testing the Web API Project with Postman
- Testing the Web API Project with Swagger
- Unit Testing in Web API
- Building First ASP.NET Core Web API

Detailed Notes

- Unit tests validate small units of behavior in isolation. A useful test has clear arrange, act, and assert phases, and it should fail for one understandable reason.
- xUnit and NUnit both support facts/tests, setup/teardown, parameterized data, assertions, fixtures, and parallel execution. Know the vocabulary even if you use one framework more often.
- Mocking is used when a class depends on external systems such as databases, APIs, queues, or file systems. The test should verify business behavior, not re-test the framework.

Important Details

- API versioning lets clients migrate gradually. Common approaches include URL segment, header, or query versioning.
- Swagger is excellent for discovery and manual testing, but automated integration tests are still needed.

How to Practice

- Write three tests: success case, validation failure, and exception/edge case.

Interview Angle

- Explain the difference between unit, integration, and end-to-end testing.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 57: Introduction to Azure DevOps

Topic family: Azure DevOps

Concepts to Cover

- Introduction to Azure DevOps
- Why Azure DevOps?
- Components of Azure DevOps
- Azure DevOps Repos
- Azure DevOps Boards Azure DevOps Pipelines
- Azure DevOps Artifacts
- Azure DevOps Test Plans

Detailed Notes

- Azure DevOps is a platform for planning work, storing code, building and releasing applications, sharing packages, and managing test activity.
- A CI/CD pipeline should restore, build, test, package, and deploy with clear environment variables and secrets. Manual deployment should become the exception, not the default.
- Boards, Repos, Pipelines, Artifacts, and Test Plans are connected pieces of a delivery workflow, not separate tools to memorize in isolation.

Important Details

- Azure DevOps connects planning, source control, CI/CD, artifacts, and test management into one delivery flow.

How to Practice

- Sketch a CI pipeline that restores, builds, tests, and publishes an ASP.NET Core app.

Interview Angle

- Expect components of Azure DevOps and what a pipeline does.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 58: Cloud Computing & Microsoft Azure Fundamentals

Topic family: Cloud Computing & Azure Fundamentals

Concepts to Cover

- Cloud Computing & Microsoft Azure Fundamentals
- Overview of Cloud Computing
- Benefits
- CapEx / OpEx
- Overview of Public Cloud
- Overview of Private Cloud
- Overview of Hybrid Cloud

Detailed Notes

- Cloud computing trades local ownership for managed capacity. You pay for usage, gain elasticity, and take responsibility for correct architecture and cost control.
- Public, private, and hybrid cloud models differ by ownership and access. Azure provides compute, storage, networking, identity, databases, monitoring, and governance services.
- Resource groups, subscriptions, roles, policies, and regions are governance concepts. They decide who can create resources, where they live, and how cost/security is managed.

Important Details

- CapEx buys long-lived assets upfront. OpEx pays for usage over time, which is a core cloud financial model.

How to Practice

- Draw the Azure resources needed for a simple web app and explain ownership, cost, and security.

Interview Angle

- Expect CapEx vs OpEx, public/private/hybrid cloud, resource groups, RBAC, and Azure portal basics.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 59: Azure Platform & Core Services

Topic family: Cloud Computing & Azure Fundamentals

Concepts to Cover

- Azure Platform & Core Services
- Microsoft Azure Portal Overview
- Utilization
- Cost
- Core Azure Service
- Availability Zones
- Availability Set
- Resource Groups
- Azure Resource Manager

Detailed Notes

- Cloud computing trades local ownership for managed capacity. You pay for usage, gain elasticity, and take responsibility for correct architecture and cost control.
- Public, private, and hybrid cloud models differ by ownership and access. Azure provides compute, storage, networking, identity, databases, monitoring, and governance services.
- Resource groups, subscriptions, roles, policies, and regions are governance concepts. They decide who can create resources, where they live, and how cost/security is managed.

Important Details

- HTML forms submit key-value pairs. In ASP.NET model binding, input name attributes must match parameter or property names.
- Resource groups are lifecycle containers for Azure resources. Use naming, tags, and ownership consistently.

How to Practice

- Draw the Azure resources needed for a simple web app and explain ownership, cost, and security.

Interview Angle

- Expect CapEx vs OpEx, public/private/hybrid cloud, resource groups, RBAC, and Azure portal basics.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 60: Security & Governance Overview

Topic family: Cloud Computing & Azure Fundamentals

Concepts to Cover

- Security & Governance Overview
- Identity
- Azure Active Directory
- Users & Groups
- Subscriptions and Accounts
- Azure Policy
- Role-Based Access Control (RBAC)

Detailed Notes

- Cloud computing trades local ownership for managed capacity. You pay for usage, gain elasticity, and take responsibility for correct architecture and cost control.
- Public, private, and hybrid cloud models differ by ownership and access. Azure provides compute, storage, networking, identity, databases, monitoring, and governance services.
- Resource groups, subscriptions, roles, policies, and regions are governance concepts. They decide who can create resources, where they live, and how cost/security is managed.

Important Details

- RBAC grants permissions to identities at scopes such as management group, subscription, resource group, or resource.

How to Practice

- Draw the Azure resources needed for a simple web app and explain ownership, cost, and security.

Interview Angle

- Expect CapEx vs OpEx, public/private/hybrid cloud, resource groups, RBAC, and Azure portal basics.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 61: Azure Portal and Cloud Shell

Topic family: Cloud Computing & Azure Fundamentals

Concepts to Cover

- Azure Portal and Cloud Shell
- Azure PowerShell
- Azure CLI
- Azure Key Vault Services
- Introduction of ARM Templates

Detailed Notes

- Cloud computing trades local ownership for managed capacity. You pay for usage, gain elasticity, and take responsibility for correct architecture and cost control.
- Public, private, and hybrid cloud models differ by ownership and access. Azure provides compute, storage, networking, identity, databases, monitoring, and governance services.
- Resource groups, subscriptions, roles, policies, and regions are governance concepts. They decide who can create resources, where they live, and how cost/security is managed.

Important Details

- Key Vault keeps secrets out of source code. Prefer managed identity so apps do not need stored credentials.

How to Practice

- Draw the Azure resources needed for a simple web app and explain ownership, cost, and security.

Interview Angle

- Expect CapEx vs OpEx, public/private/hybrid cloud, resource groups, RBAC, and Azure portal basics.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 62: Azure Virtual Machine

Topic family: Cloud Networking & Storage

Concepts to Cover

- Azure Virtual Machine
- Create Virtual Machine
- Manage Virtual Machine
- Create VM Images
- Create Scale Set
- Introduction to Load Balancer VMs

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- VMs provide infrastructure control but require patching, monitoring, security, and cost management.
- Load balancers distribute traffic across backend instances and improve availability.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 63: Overview of Virtual Network

Topic family: Cloud Networking & Storage

Concepts to Cover

- Overview of Virtual Network
- Filter Network Traffic – NSG
- Secure Network Traffic
- Application Security Groups
- Difference between NSG and ASG

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Filters run before or after actions and can handle authorization, resource checks, exceptions, and result processing.
- NSGs filter traffic using rules. Avoid broad inbound rules such as any source to RDP/SSH.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 64: Virtual Network Service Endpoints

Topic family: Cloud Networking & Storage

Concepts to Cover

- Virtual Network Service Endpoints
- Private Endpoint
- Virtual Network Peering
- Create & Manage VPN Gateway
- Create Site to Site VPN Connection
- Gateway Configuration Settings

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Private endpoints expose Azure services through private IPs inside a VNet.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 65: Azure Application Gateway

Topic family: Cloud Networking & Storage

Concepts to Cover

- Azure Application Gateway
- Create Application Gateway
- Support High Traffic Volumes
- Autoscaling and Zone-redundant Application Gateways
- Azure CDN
- Create Azure CDN Profile and Endpoint
- Monitor Health of Azure CDN Resources
- Azure Diagnostic Logs
- Azure CDN Usage Patterns

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 66: Azure Storage Services – semi structured & non-structured data

Topic family: Cloud Networking & Storage

Concepts to Cover

- Azure Storage Services – semi structured & non-structured data
- Core Storage Services
- Azure Blobs – Binary Large Object
- Azure Files – Network File Share
- Azure Queues – Asynchronous Communication
- Azure Tables – Semi Structured (Key–Value)
- Azure Disks

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Async/await is best for I/O-bound work. It does not automatically make CPU-bound work faster, but it prevents blocking request threads.
- Blob Storage is designed for unstructured objects such as images, documents, backups, and logs.
- Queues decouple producers and consumers. They improve resilience when downstream processing is slow or temporarily unavailable.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 67: Creating Storage Account

Topic family: Cloud Networking & Storage

Concepts to Cover

- Creating Storage Account
- Azure Blob Storage
- Work with Blobs
- Upload, Download & List Blobs
- Encrypt & Decrypt Blobs using Azure Key Vault
- Authorize Access to Azure Storage

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Key Vault keeps secrets out of source code. Prefer managed identity so apps do not need stored credentials.
- Blob Storage is designed for unstructured objects such as images, documents, backups, and logs.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 68: Choosing Data Storage Technology in Azure

Topic family: Cloud Networking & Storage

Concepts to Cover

- Choosing Data Storage Technology in Azure
- Structured Data
- Semi Structured Data
- Unstructured Data
- Accessing Azure Storage using Azure SDK & C#.NET

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 69: Azure API Management

Topic family: Cloud Networking & Storage

Concepts to Cover

- Azure API Management
- API Management Features
- Create an Instance
- Manage API Management
- Import and Publish First API
- Mock API Response
- Monitor Published APIs
- Self-Hosted Gateway Overview
- Overview of Products
- Creating and Publishing Products
- Add an API to an Existing Product

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Mock external dependencies such as repositories, email senders, queues, and HTTP clients. Do not mock simple value objects.
- API Management adds gateway features such as policies, products, subscriptions, transformations, mock responses, and monitoring.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 70: Azure Web App

Topic family: Azure PaaS Services

Concepts to Cover

- Azure Web App
- What is App Service
- App Service Plan
- Comparison between App Service Plans
- App Service Environments

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- App Service hosts web apps and APIs without managing servers. App Service Plan controls compute and cost.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 71: Creating ASP.NET Web App

Topic family: Azure PaaS Services

Concepts to Cover

- Creating ASP.NET Web App
- Deploying App Using Visual Studio & Kudu
- Run App in Staged Environments using Deployment Slots
- Working with Configurations
- Accessing Configuration using ASP.NET Application

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- Deployment is not only publishing files. You must decide runtime, hosting model, environment configuration, database connectivity, logging, and rollback approach.
- Deployment slots allow staging and swap-based releases with safer rollback.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 72: Host API with CORS

Topic family: Azure PaaS Services

Concepts to Cover

- Host API with CORS
- Creating and Using Web Jobs
- Ways to Authenticate App Services in Azure Azure Function App
- Overview of Serverless Computing
- Benefits of Serverless Computing
- Serverless Comparison
- Hosting Plan

Detailed Notes

- A Web API exposes application behavior over HTTP. REST-style APIs use resources, standard verbs, status codes, request/response bodies, and predictable routes.
- DTOs are API contracts. They prevent overposting, hide database-only fields, and allow the API to evolve separately from EF entities.
- Cross-cutting concerns such as validation, error handling, logging, security, CORS, caching, and versioning should be designed deliberately instead of sprinkled randomly.

Important Details

- CORS is enforced by browsers. Configure only trusted origins, methods, and headers.
- App Service hosts web apps and APIs without managing servers. App Service Plan controls compute and cost.
- Functions are event-driven and scale based on triggers. Keep functions focused and idempotent where possible.

How to Practice

- Design REST endpoints for one resource and test them through Swagger and Postman.

Interview Angle

- Expect REST, status codes, DTOs, CORS, Swagger, authentication, logging, and versioning.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 73: Email communication

Topic family: Power Skill 4

Concepts to Cover

- Email communication
- Objective setting on written communication
- 7C's (Email etiquette)
- Email writing Test and activities
- Accountability & Ownership
- Teamwork

Detailed Notes

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.
- Practice STAR stories for behavioral interviews: Situation, Task, Action, Result. Use real training examples such as debugging, teamwork, deadlines, and learning a new tool.

Important Details

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.

How to Practice

- Prepare one written email, one 2-minute presentation, and one STAR story from your training.

Interview Angle

- Expect email etiquette, presentation structure, teamwork, ownership, and behavioral examples.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 74: Create Function App using C# in Azure Portal

Topic family: Azure PaaS Services

Concepts to Cover

- Create Function App using C# in Azure Portal
- Deploying Function App using Visual Studio
- Bindings & Triggers and Their Types
- Implementing Various Triggers
- Accessing Functions using Keys
- Durable Functions

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- Functions are event-driven and scale based on triggers. Keep functions focused and idempotent where possible.
- Durable Functions orchestrate stateful workflows using orchestrator, activity, entity, and client functions.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 75: Azure Logic App

Topic family: Azure PaaS Services

Concepts to Cover

- Azure Logic App
- Introduction to Logic App
- Single Tenant versus Multi-Tenant
- Creating Logic App
- Schedule-Based Workflows
- Approval-Based Workflows

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- Logic Apps are workflow automation services with connectors and visual design, useful for integration scenarios.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 76: Creating Azure Storage and Azure Function Workflow

Topic family: Azure PaaS Services

Concepts to Cover

- Creating Azure Storage and Azure Function Workflow
- Deploy Logic Apps
- Automated Logic App Deployment

Detailed Notes

- Azure networking controls how resources talk to each other and to the internet. VNets, subnets, NSGs, private endpoints, peering, gateways, and load balancers are the core building blocks.
- Azure Storage covers blobs, files, queues, tables, and disks. Choose storage based on data shape, access pattern, durability, cost, and integration needs.
- Security and cost should be part of every design: restrict public access, prefer private connectivity, monitor logs, choose the right redundancy tier, and stop unused compute.

Important Details

- Deployment is not only publishing files. You must decide runtime, hosting model, environment configuration, database connectivity, logging, and rollback approach.
- Functions are event-driven and scale based on triggers. Keep functions focused and idempotent where possible.
- Logic Apps are workflow automation services with connectors and visual design, useful for integration scenarios.

How to Practice

- Design a secure storage/network setup using private access where possible.

Interview Angle

- Expect VNets, subnets, NSGs, private endpoints, Blob/File/Queue/Table storage, and cost/security choices.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 77: Azure Database Services

Topic family: Azure PaaS Services

Concepts to Cover

- Azure Database Services
- Database Workloads in Azure
- OLAP in Azure – Introduction
- Non-Relational Databases in Azure
- Azure SQL Introduction
- Migrate to Azure SQL

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 78: SQL Managed Instances

Topic family: Azure PaaS Services

Concepts to Cover

- SQL Managed Instances
- Elastic Pools
- Instance Pools
- Create SQL Database
- Configure Firewall
- Configuring Security
- Logins, User Accounts, Roles and Permissions

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 79: Cosmos DB

Topic family: Azure PaaS Services

Concepts to Cover

- Cosmos DB
- Introduction to Azure Cosmos DB
- NoSQL vs Relational Databases
- Cosmos DB Resource Model
- Global Distribution
- Partitioning and Horizontal Scaling
- Create an Azure Cosmos Account
- Build a .NET Web App to Manage Data
- Query Data with SQL Queries
- Introduction to Azure Cosmos DB Cassandra API
- Types of Consistencies in Cosmos DB
- Ways to authenticate App Services in Azure

Detailed Notes

- PaaS services reduce infrastructure responsibility. App Service, Function Apps, Logic Apps, Azure SQL, and Cosmos DB let teams focus more on application code and workflows.
- App Service plans control compute for Web Apps. Deployment slots help safe releases. Configuration should come from environment settings or Key Vault, not hard-coded source.
- For data services, know the trade-off between relational Azure SQL and globally distributed NoSQL Cosmos DB. The choice depends on schema, consistency, partitioning, and query needs.

Important Details

- App Service hosts web apps and APIs without managing servers. App Service Plan controls compute and cost.
- Cosmos DB performance depends strongly on partition key choice and RU consumption. Poor partition design is expensive.

How to Practice

- Deploy or diagram a Web App/Function plus database flow and list configuration settings.

Interview Angle

- Expect App Service, Functions, Logic Apps, Azure SQL, Cosmos DB, deployment slots, and configuration.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 80: Team work and accountability

Topic family: Power Skill 5

Concepts to Cover

- Team work and accountability
- Listening, Questioning &
- Probing
- Meeting Etiquette
- Interview Skills
- Comprehensive Test (Email, Grammar, Behavioural)

Detailed Notes

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.
- Practice STAR stories for behavioral interviews: Situation, Task, Action, Result. Use real training examples such as debugging, teamwork, deadlines, and learning a new tool.

Important Details

- Power skills are employability skills: communication, email writing, listening, presentation, ownership, teamwork, meeting behavior, and interview readiness.
- Technical work is judged through communication. Clear status updates, concise explanations, and structured emails often decide whether your work is trusted.

How to Practice

- Prepare one written email, one 2-minute presentation, and one STAR story from your training.

Interview Angle

- Expect email etiquette, presentation structure, teamwork, ownership, and behavioral examples.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 81: Getting Started with Git

Topic family: Git

Concepts to Cover

- Getting Started with Git
- Install the Git Tools
- Clone an Existing Repository
- Add Files to a Repository
- Edit Files in a Git Repository

Detailed Notes

- Git tracks snapshots of source code. The working tree holds current files, the staging area prepares the next commit, and commits record project history.
- Branches isolate work. Pull requests create a review and integration point. Merge conflicts happen when changes touch the same lines and must be resolved intentionally.
- Good Git hygiene means small commits, clear messages, no secrets, no generated build folders, and regular pulls from the shared branch.

Important Details

- Repository pattern hides data access details behind an interface. It is useful when it simplifies testing and isolates persistence logic.

How to Practice

- Create a branch, make a commit, merge it, and resolve a planned conflict.

Interview Angle

- Expect branch, merge, rebase, conflict resolution, stash, reset, revert, and pull request workflow.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Syllabus Focus Sheet 82: Create and Merge Branches

Topic family: Git

Concepts to Cover

- Create and Merge Branches
- Rewrite History in a Git Repository
- Resolve Merge Conflicts

Detailed Notes

- Git tracks snapshots of source code. The working tree holds current files, the staging area prepares the next commit, and commits record project history.
- Branches isolate work. Pull requests create a review and integration point. Merge conflicts happen when changes touch the same lines and must be resolved intentionally.
- Good Git hygiene means small commits, clear messages, no secrets, no generated build folders, and regular pulls from the shared branch.

Important Details

- Repository pattern hides data access details behind an interface. It is useful when it simplifies testing and isolates persistence logic.
- Resolve merge conflicts by understanding both changes, editing the conflicted file, staging it, and committing the resolution.

How to Practice

- Create a branch, make a commit, merge it, and resolve a planned conflict.

Interview Angle

- Expect branch, merge, rebase, conflict resolution, stash, reset, revert, and pull request workflow.

Self-Test

- Can I explain this without reading the notes?
- Can I build or write a tiny example in less than 15 minutes?
- Can I identify one production risk or best practice related to it?

Final Sprint Readiness Checklist

- I can create, build, run, and explain a .NET 8 project.
- I can explain C# types, OOP, exceptions, collections, delegates, LINQ, async, and C# 12 features.
- I can solve basic algorithm questions and discuss complexity.
- I can write xUnit/NUnit tests with assertions and parameterized data.
- I can design SQL tables with keys and relationships, write joins, create views/indexes/procedures, and read execution plans at a basic level.
- I can use EF Core with DbContext, migrations, Code First, DB First, Include, tracking, and repository pattern.
- I can build HTML/CSS/JavaScript forms and explain DOM events, validation, fetch, and storage.
- I can build MVC CRUD using controllers, views, view models, validation, layouts, partial views, and filters.
- I can design Web APIs with REST routes, DTOs, status codes, Swagger, CORS, authentication, logging, caching, and versioning.
- I can explain Azure DevOps Boards, Repos, Pipelines, Artifacts, and Test Plans.
- I can choose Azure services for compute, storage, networking, identity, App Service, Functions, Logic Apps, Azure SQL, and Cosmos DB.
- I can use Git branch, merge, conflict resolution, history inspection, and pull request workflow.
- I can present my project clearly, write professional emails, and answer behavioral questions with STAR.